Labo 17 — Questions

Question 1 [Compréhension]

Expliquez la différence entre un chart, une release et une revision. Où Helm 3/4 stocke-t-il l'historique d'une release, et pourquoi cela permet-il `helm rollback` sans aucun composant serveur ?

Question 2 [Prédiction]

```
helm install stack ./stack -n lab17 --create-namespace
helm upgrade stack ./stack --set api.replicas=3
helm upgrade stack ./stack --set api.message="v2"
helm get values stack
```

Que montre la dernière commande ? Combien de réplicas tourne l'API après la troisième commande, et pourquoi ? Que faudrait-il changer pour obtenir à la fois trois réplicas et le message « v2 » ?

Question 3 [Diagnostic]

```
$ helm list -A
Error: kubernetes cluster unreachable: Get "http://localhost:8080/version": dial tcp
[::1]:8080: connect: connection refused
$ kubectl get nodes
NAME         STATUS   ROLES           AGE   VERSION
wsl-ubuntu   Ready    control-plane   6d    v1.34.1+k3s1
```

Le cluster tourne, `kubectl` le voit, `helm` non. Expliquez, donnez la correction, et dites pourquoi `kubectl` n'a pas le même problème.

Question 4 [Analyse]

Le chart annote le template de Pod avec `checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}`. Expliquez ce que cela produit à `helm upgrade --set api.message=autre`, ce qui se passerait sans cette annotation (labo 14), et un cas où cet idiome déclenche un redéploiement non souhaité.

Question 5 [Prédiction]

```
helm history stack          # revisions 1, 2, 3 ; 3 = deployed
helm rollback stack 1
helm history stack
helm uninstall stack
kubectl get pvc -n lab17
```

Décrivez l'historique après le rollback (nombre de revisions, description de la dernière), puis ce que montre `kubectl get pvc`. Pourquoi Helm se comporte-t-il ainsi avec les PVC, et quel geste supprime vraiment les données ?

Question 6 [Analyse]

Comparez Helm et Kustomize : modèle, distribution, historique, cas d'usage. Une équipe doit (a) installer PostgreSQL et Prometheus fournis par des tiers, (b) déployer ses quatre microservices dans trois environnements via Argo CD. Que choisiriez-vous pour chaque besoin, et pourquoi les deux cohabitent-ils souvent ?

Question 7 [Diagnostic]

```
# overlays/prod/kustomization.yaml
resources: [../../base]
images:
  - name: localhost/api
    newTag: "1.1"
labels:
  - pairs: { env: prod }
    includeSelectors: false
```

Après `kubectl apply -k overlays/prod`, les Pods de l'API portent bien l'image 1.1 mais **pas** le label `env=prod` ; le Deployment, lui, l'a. Expliquez, corrigez, et dites pourquoi `includeSelectors: true` sur un Deployment existant serait une mauvaise idée.

Question 8 [Analyse]

`podman image inspect api:1.1 --format '{{.Digest}}'` affiche `sha256:e1b2…` ; `podman push --digestfile` écrit `sha256:b820…` ; `kubectl get pod -o jsonpath='{..imageID}'` montre `localhost:5000/api@sha256:b820…`. Expliquez pourquoi deux digests différents désignent la même image, lequel il faut épingler en production, et pourquoi un tag ne suffit pas.

Question 9 [Diagnostic]

Un Pod reste en `ImagePullBackOff` avec `failed to pull image "registry.interne:5000/api:1.1": … http: server gave HTTP response to HTTPS client`. Le même registry marche en `localhost:5000`. Expliquez la différence de traitement, donnez le fichier et le contenu à écrire sur k3s, et ce qu'il faut faire après. Que ferait-on sur un cluster managé où l'on n'a pas accès aux nœuds ?

Question 10 [Analyse]

Un pipeline exécute `helm upgrade --install api ./chart --set api.image.tag=$CI_COMMIT_SHA --wait --timeout 5m` avec un kubeconfig. Dites ce que `--install`, `--wait` et `--timeout` apportent, quel kubeconfig le runner devrait avoir (labo 16), et ce qu'il devrait faire si la commande échoue.

Question 11 [Compréhension]

Expliquez le GitOps en opposant « la CI pousse vers le cluster » et « le cluster tire depuis Git ». Quels avantages concrets (audit, rollback, dérive) et quelle conséquence sur ce que la CI a encore le droit de faire ? Reliez cela à la boucle de réconciliation du labo 11.

Question 12 [Analyse]

Votre chart fonctionne sur k3s avec `Service: LoadBalancer`, `local-path` et l'Ingress Traefik. Listez ce qui changera concrètement en passant sur EKS/AKS/GKE, puis sur OpenShift, et ce qui ne changera pas. Comment écrire le chart pour qu'il reste portable ?

Question 13 [Analyse]

Un développeur propose : « on met `latest` partout, `imagePullPolicy: Always`, et la CI fait `kubectl rollout restart` après chaque push : plus besoin de changer de tag ni de chart ». Démontez la proposition point par point (reproductibilité, rollback, audit, sécurité), puis proposez la version correcte avec les outils de ce labo.